

Tarea 3

Implementamos un algoritmo mediante la integración de $\frac{4}{x^2+1}$ de 0 a 1 para encontrar el valor π .
Comparamos los tiempos de ejecución de la implementación serial y en OpenMPI.

Índice

1. Introducción	1
2. Implementaciones	2
2.1. Secuencial	2
2.1.1. Compilación del programa	2
2.1.2. Ejecución del programa	2
2.1.3. Ejemplos	2
2.2. OpenMPI	3
2.2.1. Compilación del programa	3
2.2.2. Ejecución del programa	3
2.2.3. Ejemplo	3
3. Análisis de resultados	4
4. Conclusiones	6

1. Introducción

La constante π tiene un valor aproximado de 3.14159.... Es un número trascendental. Esto quiere decir que no existe una ecuación polinomial para la cual $P(\pi) = 0$. Entonces, debemos encontrar maneras creativas de aproximar el valor de esta constante.

Recordemos que

$$\frac{d}{dt} \arctan(t) = \frac{1}{1+t^2},$$

por lo que

$$\int_0^1 \frac{1}{1+t^2} dt = \arctan(1) - \arctan(0) = \frac{\pi}{4}.$$

Entonces, una manera de aproximar el valor de π es calculando una aproximación a la integral

$$\int_0^1 \frac{1}{1+t^2} dt.$$

Para aproximar esta integral, podemos utilizar diversos métodos numéricos. Por simplicidad, usaremos el método del rectángulo. Es decir, dada la curva $y(t) = \frac{1}{1+t^2}$, dividimos el intervalo $[0, 1]$ en N trozos de la misma medida. Suponemos que los límites de cada trozo es x_i para $i = 0, \dots, N$. Luego, formamos el rectángulo

$$[x_i, x_{i+1}] \times \left[\min\left(f\left(\frac{x_i + x_{i+1}}{2}\right), 0\right), \max\left(0, f\left(\frac{x_i + x_{i+1}}{2}\right)\right) \right]$$

Es decir, un rectángulo con una altura del punto medio de los límites de la partición. Notemos que estos rectángulos también pueden ir hacia abajo del eje x .

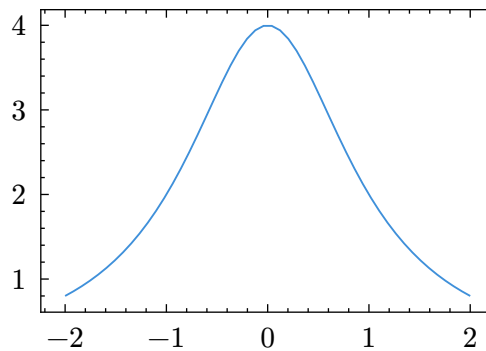


Figura 1: Función $\frac{4}{1+x^2}$ en $[-2, 2]$.

2. Implementaciones

Presentamos partes cruciales del código de ambas implementaciones.

2.1. Secuencial

Para la versión secuencial, debemos recorrer cada uno de los rectángulos y realizar la suma del área de cada uno.

```
double step = (b - a) / static_cast<double>(N);  
double area = 0.0;  
double curr_mark = a;  
for (long i = 0; i < N; i++)  
{  
    double mid = curr_mark + step / 2;  
    area += step * f(mid);  
    curr_mark += step;  
}  
return area;
```

Listado 1: Recorrido secuencial para la suma de áreas.

2.1.1. Compilación del programa

Para compilar el programa, usamos

```
g++ integral_seq.cpp -o seq
```

Listado 2: Compilación de la implementación secuencial.

2.1.2. Ejecución del programa

Para ejecutar el programa, usamos

```
./seq <N>
```

Listado 3: Ejecución del programa. Los argumentos de línea corresponden a N : número de particiones del intervalo.

2.1.3. Ejemplos

```
./seq 1000
```

Listado 4: Ejemplo con $N = 1000$.

El output de esta ejecución es

```
===== SEQUENTIAL =====  
Execution time: 0.006615 ms  
Area: 3.14159273692313  
Error: 8.33333366756506e-08  
Listado 5: Ejemplo con  $N = 1000$ .
```

2.2. OpenMPI

```
double step = (b - a) / static_cast<double>(N);  
double global_area = 0.0;  
double area = 0.0;  
long start = taskid * N / numtasks;  
long end = (taskid + 1) * N / numtasks;  
double curr_mark = a + start * step;  
  
for (int i = start; i < end; i++)  
{  
    double mid = curr_mark + step / 2;  
    area += step * f(mid);  
    curr_mark += step;  
}  
MPI_Reduce(&area, &global_area, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);  
  
return global_area;
```

Listado 6: Recorrido paralelo usando Reduce para la suma de áreas.

2.2.1. Compilación del programa

Para compilar el programa, usamos

```
mpicxx integral_mpi.cpp -o mpi
```

Listado 7: Compilación de la implementación en OpenMPI.

2.2.2. Ejecución del programa

Para ejecutar el programa, usamos

```
mpirun -n <processes> ./mpi <N>
```

Listado 8: Ejecución del programa. Los argumentos de línea corresponden a N : número de particiones del intervalo.

2.2.3. Ejemplo

```
mpirun -n 4 ./mpi 1000
```

Listado 9: Ejemplo con $N = 1000$

El output de esta ejecución es

```
===== MPI_REDUCE =====  
Execution time: 0.143244 ms  
Area: 3.14159273692313  
Error: 8.33333331229369e-08  
===== MPI_SEND =====  
Execution time: 0.005472 ms  
Area: 3.14159273692313  
Error: 8.33333335670261e-08  
Listado 10: Ejemplo con  $N = 1000$ .
```

3. Análisis de resultados

N	Versión	Tiempo (ms)	Aproximación	Error
1,000	Secuencial	0.007800	3.14159273692313	8.33333366756506e-08
	Reduce	0.013100	3.14159273692313	8.33333366756506e-08
	Send	0.006300	3.14159273692313	8.33333366756506e-08
	AllGather	0.008700	3.14159273692313	8.33333366756506e-08
10,000	Secuencial	0.065600	3.14159265442323	8.33440427783216e-10
	Reduce	0.066100	3.14159265442323	8.33440427783216e-10
	Send	0.094400	3.14159265442323	8.33440427783216e-10
	AllGather	0.122000	3.14159265442323	8.33440427783216e-10
1,000,000	Secuencial	6.694400	3.14159265359130	1.50945922428036e-12
	Reduce	5.887600	3.14159265359130	1.50945922428036e-12
	Send	6.176702	3.14159265359130	1.50945922428036e-12
	AllGather	7.837703	3.14159265359130	1.50945922428036e-12

Tabla 1: Rendimiento comparativo entre ejecución secuencial y distintas versiones de MPI para 1 proceso.

N	Versión	Tiempo (ms)	Aproximación	Error
1,000	Reduce	0.612202	3.14159273692313	8.33333366756506e-08
	Send	0.005400	3.14159273692313	8.33333366756506e-08
	AllGather	0.023200	3.14159273692313	8.33333366756506e-08
10,000	Reduce	0.112198	3.14159265442317	8.33380919829096e-10
	Send	0.034699	3.14159265442317	8.33380919829096e-10
	AllGather	0.035599	3.14159265442317	8.33380919829096e-10
1,000,000	Reduce	3.151810	3.14159265358348	6.30961949354969e-12
	Send	3.909611	3.14159265358348	6.30961949354969e-12
	AllGather	6.692720	3.14159265358348	6.30961949354969e-12

Tabla 2: Rendimiento comparativo entre distintas versiones de MPI para 2 procesos.

N	Versión	Tiempo (ms)	Aproximación	Error
1,000	Reduce	0.045927	3.14159273692313	8.33333331229369e-08
	Send	0.005472	3.14159273692313	8.33333335670261e-08
	AllGather	0.009533	3.14159273692313	8.33333335670261e-08
10,000	Reduce	0.100905	3.14159265442316	8.33363156260702e-10
	Send	0.028338	3.14159265442316	8.33362712171493e-10
	AllGather	0.028876	3.14159265442316	8.33362712171493e-10
1,000,000	Reduce	1.624320	3.14159265358765	2.14450679436595e-12
	Send	1.734390	3.14159265358765	2.14450679436595e-12
	AllGather	1.664277	3.14159265358765	2.14450679436595e-12

Tabla 3: Rendimiento comparativo entre distintas versiones de MPI para 4 procesos.

N	Versión	Tiempo (ms)	Aproximación	Error
1,000	Reduce	0.185	3.14159273692313	8.33333335670261e-08
	Send	0.004900	3.14159273692313	8.33333331229369e-08
	AllGather	0.008900	3.14159273692313	8.33333331229369e-08
10,000	Reduce	0.849599	3.14159265442314	8.33347613138358e-10
	Send	0.060000	3.14159265442314	8.33348057227568e-10
	AllGather	0.021400	3.14159265442314	8.33348057227568e-10
1,000,000	Reduce	1.998700	3.14159265358870	1.09512399149025e-12
	Send	2.209100	3.14159265358870	1.09467990228040e-12
	AllGather	1.258800	3.14159265358870	1.09467990228040e-12

Tabla 4: Rendimiento comparativo entre distintas versiones de MPI para 8 procesos.

Por lo observado, usar demasiados procesos ralentiza el tiempo de ejecución. Esto puede ser por la cantidad de comunicación que debe hacerse para un problema simple como este.

Podemos observar que los errores difieren entre cada versión. Esto es completamente esperado, ya que la precisión de punto flotante hace que los resultados sean distintos. Para la versión secuencial, los valores se suman de la siguiente manera:

$$((R1 + R2) + R3) + R4 + \dots$$

Listado 11: Suma de áreas de rectángulos en la versión secuencial.

$$(A1 + A2) + (A3 + A4) + \dots$$

Listado 12: Suma de áreas de rectángulos en la versión Reduce. La suma se hace en manera de árbol según los procesos. Cada A representa el área que cubre cada proceso. Entonces, el error de punto flotante es distinto al secuencial, ya que no se sumaron de manera lineal.

$$((A1 + A2) + A3) + A4 + \dots$$

Listado 13: Suma de áreas de rectángulos en la versión Send. Notemos que aquí difiere de la versión secuencial porque las A representan las áreas obtenidas de cada proceso.

$$((A1 + A2) + A3) + A4 + \dots)$$

Listado 14: Suma de áreas de rectángulos en la versión Allgather. La suma es igual al Send, por lo que esperamos un error idéntico.

4. Conclusiones

Cada ejecución resultó en un error distinto. Esto se explica por el orden en el que se sumaron las áreas de cada rectángulo. Los tiempos obtenidos indican que las versiones paralelas tienen un mejor rendimiento que la secuencial. Tanto Reduce como Send con Receive y Allgather tienen un desempeño similar para estos valores. Sin embargo, la complejidad de Reduce es de $O(\log N)$, ya que la suma se hace a manera de árbol. Esta diferencia se notaría más con el uso de un mayor número de procesos.